```
Reinforcement Learning
        │
        └── Multi-Armed Bandit Problem
                │
                ├── Action-Value Methods
                │       │
                │       ├─ Greedy Action Selection
                │       │
                │       ├─ ε-Greedy Method
                │       │
                │       ├─ Incremental Implementation
                │       │
                │       ├─ Non-Stationary Methods
                │       │       └─ Constant Step Size (α)
                │       │
                │       ├─ Optimistic Initial Values (OIV)
                │       │
                │       └─ Upper Confidence Bound (UCB)
                │
                ├── Preference-Based Methods
                │       └─ Gradient Bandits
                │
                └── Contextual Bandits
                        └─ Associative Search
```

The n-Armed Bandit Problem

The **n-armed bandit problem** is the **simplest form of a reinforcement learning problem**.

- In this problem, an agent repeatedly chooses among **n different actions** and receives a **numerical reward** after each selection. The reward received depends on the action chosen and follows an **unknown probability distribution**.
- The objective of the agent is to **maximize the expected cumulative reward over a sequence of time steps** (for example, over 1000 action selections).

The **n-armed bandit problem** gets its name from a **slot machine**, which is also called a **one-armed bandit**.

- A slot machine has **one lever** that produces a random payoff when pulled.
- In the **n-armed bandit**, the machine is imagined to have **multiple levers**.

Each lever represents:

- **Action → Lever**
- **Action selection → Pulling the lever**
- **Reward → Jackpot or payoff**

The goal is to **discover which lever gives the highest reward** and select it more often.

example is a **doctor choosing treatments for patients**.

- Each **treatment** is an action.
- The **reward** is the patient's survival or health improvement.
- The doctor must determine which treatment works best by observing outcomes.

Action Value

Each action has an **expected (average) reward**, which is called the **value of the action**.

- If the true value of each action were known, the problem would be easy.
- The agent would simply **choose the action with the highest value every time**.

However:

- The **true values are unknown**.
- The agent must **estimate them based on experience**.

Discuss the conflict between exploration and exploitation in the multi-armed bandit problem.
Why is balancing exploration and exploitation important in reinforcement learning?

In the n-armed bandit problem, an agent repeatedly chooses among several possible actions and receives a reward after each selection. The goal of the agent is to maximize the **total reward over time**.

Since the true reward values of the actions are not known in advance, the agent must learn them through interaction with the environment.

While learning, the agent faces an important decision: whether to use the knowledge it already has to obtain reward or to try other actions to gain more information. This leads to the concepts of **exploitation and exploration**.

Exploitation (Greedy Action)

Exploitation refers to selecting the action that currently appears to provide the **highest expected reward** based on the agent's present knowledge. The agent maintains estimates of the values of different actions using the rewards observed so far.

- The action with the **highest estimated value** is called the **greedy action**.
- When the agent chooses this greedy action, it is exploiting its current knowledge to obtain the **maximum immediate reward**.
- Exploitation focuses on using what has already been learned rather than trying new actions.

For example, suppose an agent has three actions with the following estimated rewards:

Action	Estimated Value
A	4
B	7
C	5

In this case, **Action B** has the highest estimated value, so it is the **greedy action**. Selecting action B repeatedly is an example of exploitation because the agent is choosing the action that currently seems best.

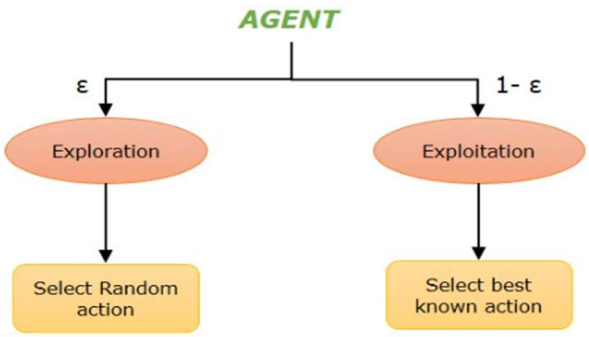
Although exploitation can produce high rewards in the short term, it may prevent the agent from discovering other actions that could potentially give **even better rewards**.

Exploration

Exploration refers to selecting an action that is **not the greedy action** in order to gain more information about its reward. Since the agent does not know the true reward values of all actions, some actions may appear worse simply because they have not been tried enough times.

- By exploring these actions, the agent can improve its estimates and possibly discover actions that give higher rewards.
- Exploration helps the agent learn more about the environment and reduce uncertainty about the value of different actions.
- Although exploration may result in **lower rewards initially**, it plays an important role in finding the truly optimal action.

Exploration–Exploitation Trade-off



A key challenge in the n-armed bandit problem is balancing **exploration and exploitation**.

Exploitation → provides higher reward in the **short term** as the agent selects the action that currently appears to be best.

Exploration → may reduce reward temporarily because the agent is trying actions that may not perform well.

However, exploration can lead to **greater rewards in the long run**.

Example:

Suppose you usually go to **Restaurant A** because you know the food is good. So choosing Restaurant A repeatedly is **exploitation** since it is the **greedy choice** based on your experience. However, there are other restaurants **B and C** that you have **never visited before**, so you do not know how good they are. If you decide to try one of them (**exploration**), you might discover that **Restaurant B is actually better than A**. After finding this, you can start going to Restaurant B regularly and get better results in the long run.

Thus, the agent must decide whether to **exploit the best known action** or **explore other actions** that might lead to better outcomes in the future.

Difficulties in Balancing Exploration and Exploitation

Balancing exploration and exploitation is difficult because the correct decision depends on several factors. These include

- the **current estimates of action values**
- the **uncertainty associated with these estimates**
- **number of remaining time steps** during which rewards can be obtained.

Various algorithms have been developed to address this problem, but many of them rely on assumptions such as stationary reward distributions or prior knowledge about the environment. In real-world applications, these assumptions may not always hold. Therefore, determining the best balance between exploration and exploitation remains a fundamental challenge in reinforcement learning.

Action-Value Methods

Action-value methods are used in the **n-armed bandit problem** to estimate the value of actions and use these estimates to select actions. Each of the **n actions** has an expected reward, which is called the **value of that action**.

Let:

- A_t = action selected at time step t
- R_t = reward received after selecting the action
- $Q_t(a)$ = estimated value of action a at time step t
- $q(a)$ = true (actual) value of action a

The **true action value** $q(a)$ is defined as the **expected reward obtained when action** a **is selected**. Since the agent does not know the true values initially, it must estimate them from experience.

<u>Estimating Action Values</u>

Sample-average method

A natural way to estimate the value of an action is by **averaging the rewards received when that action is selected**.

If action a has been selected $N_t(a)$ times before time t, producing rewards R1,R2,…,$R_{Nt(a)}$, then the estimated value of action a is:

$$Q_t(a) = \frac{R_1 + R_2 + \cdots + R_{N_t(a)}}{N_t(a)}$$

If an action has not been selected before ($N_t(a) = 0$), its value is usually initialized to a default value such as:

$$Q_1(a) = 0$$

This method is called the **sample-average method**, because the estimate of each action value is simply the **average of the observed rewards**.

According to the **law of large numbers**, as the number of times an action is selected increases ($N_t(a) \to \infty$), the estimated value $Q_t(a)$ approaches the true value $q(a)$.

Greedy Action Selection

Once the action values are estimated, they can be used to select actions. The simplest method is the **greedy action selection rule**, which selects the action with the **highest estimated value**.

$$A_t = \arg\max_a Q_t(a)$$

Here, **argmax** denotes the value of a that maximizes the expression $Q_t(a)$.

The greedy action always **exploits the current knowledge** by selecting the action that currently appears to give the highest reward. However, this method has a limitation: it does not explore other actions that might actually be better.

ε-Greedy Action Selection

To overcome this limitation, the **ε-greedy method** is used. In this approach, the agent behaves greedily most of the time, but occasionally explores other actions.

- With probability $1 - \varepsilon$ → select the greedy action
- With probability ε → select a random action from all available actions

This ensures that **all actions are explored occasionally**, allowing the agent to improve its estimates of their values.

An important advantage of ε-greedy methods is that as the number of plays increases, **every action will eventually be sampled many times**. As a result, the estimates $Q_t(a)$ converge to the true values $q(a)$, and the agent becomes more likely to select the optimal action.

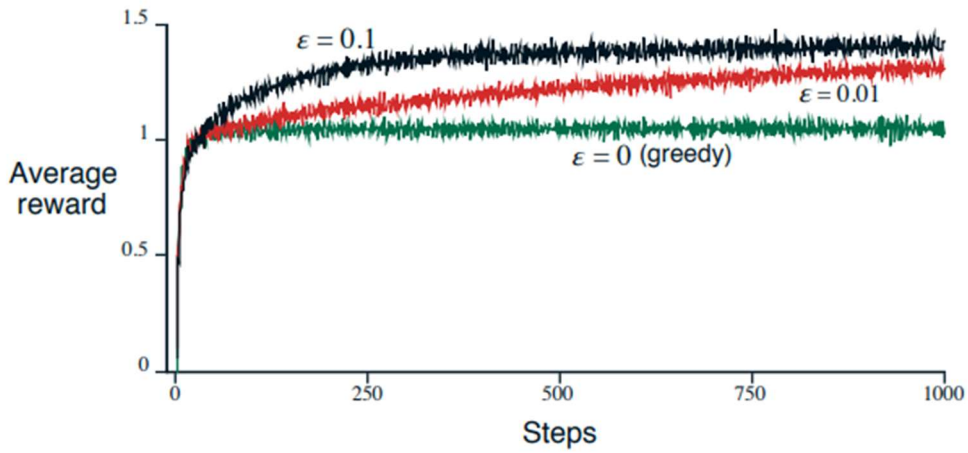
Aspect	Greedy Action Selection	ε-Greedy Action Selection
Basic Idea	Always selects the action with the **highest estimated value**.	Selects the greedy action most of the time but **occasionally explores other actions**.
Action Selection Rule	$A_t = \arg\max_a Q_t(a)$	With probability $1 - \varepsilon$: choose greedy action; with probability ε: choose a random action.
Exploration	**No exploration** – only exploitation.	**Includes exploration** by randomly selecting actions.
Goal	Maximizes **immediate reward** using current knowledge.	Balances **exploration and exploitation**.
Risk	May get **stuck with a suboptimal action** if early estimates are wrong.	Lower risk of getting stuck because it **tries other actions occasionally**.
Long-term Performance	Often performs **worse in the long run** due to lack of exploration.	Usually performs **better in the long run** because it discovers better actions.
Learning	Limited learning since other actions may never be tried.	Better learning because **all actions are eventually sampled**.
Example	Always choosing the restaurant you already know is good.	Usually choosing your favorite restaurant but **sometimes trying a new one**.

Performance Comparison (10-Armed Testbed)

To evaluate these methods, experiments were performed on a **10-armed testbed**, consisting of 2000 randomly generated bandit problems.

The experiment compares three strategies:

- Greedy method $(\varepsilon = 0)$
- ε-greedy method $(\varepsilon = 0.01)$
- ε-greedy method $(\varepsilon = 0.1)$



The **greedy method (ε = 0)**

- improves quickly at the beginning because it always selects the action with the highest estimated value.
- However, since it does not explore other actions, it often gets stuck with **suboptimal actions**, resulting in lower long-term reward.

The **ε-greedy methods (ε = 0.01 and ε = 0.1)**

- occasionally explore other actions.
- Because of this exploration, they are more likely to **discover the optimal action** and achieve higher rewards in the long run.

The method with **ε = 0.1 explores more**, so it often finds the optimal action earlier. The method with **ε = 0.01 explores less**, but eventually achieves better performance because it balances exploration and exploitation.

Incremental Implementation

In the **n-armed bandit problem**, the value of an action is estimated using the **sample average of the rewards** obtained when that action is selected.

If action a has been selected $N_t(a)$ times and the rewards obtained are $R_1, R_2, ..., R_{N_t(a)}$, then the estimated action value is

$$Q_t(a) = \frac{R_1 + R_2 + \cdots + R_{N_t(a)}}{N_t(a)}$$

This means the action value is simply the **average of all observed rewards.**

Problem with the Direct Average Method

Using the above formula directly has two major problems:

1. Memory Problem

- All past rewards must be stored.
- Memory requirement **increases over time.**

2. Computation Problem

- Each time the estimate $Q_t(a)$ is needed, we must **sum all previous rewards again.**
- Computation increases as more rewards are collected.

Therefore, this approach becomes **inefficient and unnecessary** as time progresses.

Incremental Update Idea

Instead of recomputing the average from the beginning each time
we can **update the old estimate using only the new reward**.
This approach is called **incremental implementation**.

Let

- Q_k = estimate after $k-1$ rewards
- R_k = the kth reward
- Q_{k+1} = updated estimate after receiving R_k

① Average after k rewards

$$Q_{k+1} = \frac{1}{k} \sum_{i=1}^{k} R_i$$

② Split into two terms — old rewards, new rewards

$$Q_{k+1} = \frac{1}{k} \left(R_k + \sum_{i=1}^{k-1} R_i \right)$$

③ But we know

$$Q_k = \frac{1}{k-1} \sum_{i=1}^{k-1} R_i$$

$$\sum_{i=1}^{k-1} R_i = (k-1) Q_k$$

Substute in previous equation

$$Q_{k+1} = \frac{1}{k} \left(R_k + (k-1) Q_k \right)$$

$$= \frac{1}{k} \left(R_k + k Q_k - Q_k \right)$$

$$= Q_k + \frac{1}{k} \left(R_k - Q_k \right)$$

$$\boxed{\therefore \; Q_{k+1} = Q_k + \frac{1}{k} \left(R_k - Q_k \right)}$$

$$= \quad Q_k + \frac{1}{k} \Big[R_k - Q_k \Big],$$

Step size (learning rate)

- Later learning → small steps (stable)
- Early learning → big steps

It is an error.

- If new reward is **higher than expected** → error is positive
- If new reward is **lower than expected** → error is negative
- As k increases, step size decreases → learning becomes stable

The update rule can be written in a general form:

$$\text{New Estimate} = \text{Old Estimate} + \text{Step Size} \times (\text{Target} - \text{Old Estimate})$$

Advantages of Incremental Implementation

- Uses **only one reward at a time**
- Requires **constant memory**
- Requires **constant computation**
- Uses **error-correction learning**
- Avoids recomputing averages

Tracking a Non-stationary Problem

→ We have sample average method, which estimates the value of actions by averaging all previous rewards

$$Q_{k+1} = Q_k + \frac{1}{k}(R_k - Q_k)$$

This works well in stationary environment, where true action rewards remain constant

→ In many real world situations, environment is "non-stationary" - true values of action may change overtime

For eg:

A restaurant recommendation:

- A restaurant may be very popular initially giving high customer satisfaction
- Later
  - quality of food may decrease
  - new restaurant may open nearby
- So environment becomes non-stationary.

→ As old rewards no longer represent the current situation

- It is better to give more importance to recent rewards than to older rewards.
- That is non-stationary problems require forgetting the past

Incremental Stepsize

$$Q_{k+1} = Q_k + \frac{1}{k}(R_k - Q_k)$$

$\frac{1}{k}$ becomes very small
new rewards will have no effect
Estimates will be slow to adapt
↳ believes old data more.

constant step size

$$Q_{k+1} = Q_k + \alpha(R_k - Q_k)$$

α = step size parameter
$0 < \alpha \leq 1$

large α → fast learning (noisy)
Small α → slow learning (smoother)
adjusts by moving error towards new data
↳ Recent rewards matter more.

Weighted Average of Rewards:

$$Q_{k+1} = Q_k + \alpha (R_k - Q_k)$$
$$= \alpha R_k + (1-\alpha) Q_k$$

for Q_k it can be written as,

$$Q_k = \alpha R_{k-1} + (1-\alpha) Q_{k-1}$$

Substute in Q_{k+1}

$$Q_{k+1} = \alpha R_k + (1-\alpha)(\alpha R_{k-1} + (1-\alpha) Q_{k-1})$$

Doing this Repeatedly, we get

$$Q_{k+1} = (1-\alpha)^k Q_1 + \sum_{i=1}^{k} \alpha (1-\alpha)^{k-i} R_i$$

→ Recent rewards receive higher weight
→ Older rewards receive ~~low~~ smaller weight
→ All weights add up to 1 $(\alpha(1-\alpha)^{k-i})$

Since weights decrease exponentially
↳ This is called exponential recency weighted average.

$\alpha_k(a)$ - step size used for action a at time k.

For convergence, following conditions must satisfy

$$\sum_{k=1}^{\infty} \alpha_k(a) = \infty$$

$$\sum_{k=1}^{\infty} \alpha_k^2(a) < \infty$$

Problems in Non-Stationary Setting (Very Simple)

1. Needs Continuous Exploration

If the agent becomes greedy and only picks **A**, it will **never discover that B became better**.

Example:
Earlier A was best → you keep using A
Later B becomes best → but you never try B.

---

2. No Directed Exploration

Exploration is **random**.
The agent might waste time trying **bad arms**.

---

3. Slow Detection of Sudden Changes

If B suddenly becomes very good, the agent might take time to notice because it **doesn't try B often**.

---

4. Early Rewards Can Mislead

If the **first few rewards were misleading**, the agent might think a bad arm is good.

Optimistic Initial Values

In action-value methods, the agent maintains estimates of the values of actions. These estimates depend on the **initial action-value estimates Q1(a)**

- These initial values represent the agent's **prior belief about the expected reward** of each action.
- In statistics, this effect is called **bias due to initialization**.

Usually the initial action values are set to **zero**, meaning the agent initially assumes that all actions have neutral value.

However, the initial values can also be set deliberately to **large positive numbers** in order to encourage exploration. This approach is known as the **Optimistic Initial Values (OIV) method**.

Idea of Optimistic Initial Values

In the optimistic initial values method, the agent starts with **high initial estimates for all actions**, even though the true rewards may be much smaller.

For example:

$$Q_1(a_1) = Q_1(a_2) = Q_1(a_3) = 5$$

In many bandit problems, the true action values may be around **0 or 1**, so an initial estimate of **5 is highly optimistic**.

Because the agent believes every action is very good initially, it tries different actions. When the actual reward is lower than the optimistic estimate, the estimate decreases, and the agent begins trying other actions. In this way, **optimistic initial values encourage exploration automatically** even when using a greedy policy.

Disappointment Effect

A key idea in optimistic initial values is the **disappointment effect**.

When an action is tried for the first time, the reward obtained is usually **lower than the optimistic initial estimate**. As a result, the value estimate for that action decreases.

For example:

Initial estimates:

Action	Initial Value
A	5
B	5
C	5

Suppose the agent selects **Action A** and receives reward $R = 1$. Since the reward is much smaller than the initial estimate, the updated estimate decreases. Now the untried actions **B and C still appear better**, so the agent selects them next.

Thus, each tried action becomes less attractive, while untried actions remain optimistic. This forces the agent to try **all actions several times**, ensuring systematic exploration.

Limitation in Non-Stationary Problems

Optimistic initial values work well in **stationary environments**, where the true reward values of actions remain constant over time.

However, this method is not suitable for **non-stationary problems**, where action values may change over time. The reason is that the exploration produced by optimistic initialization is **temporary**. It occurs only at the beginning of learning. If the environment changes later, the agent will not explore again and may fail to detect better actions.

In non-stationary environments, methods that perform **continuous exploration** (such as ε-greedy with constant step size) are more appropriate.

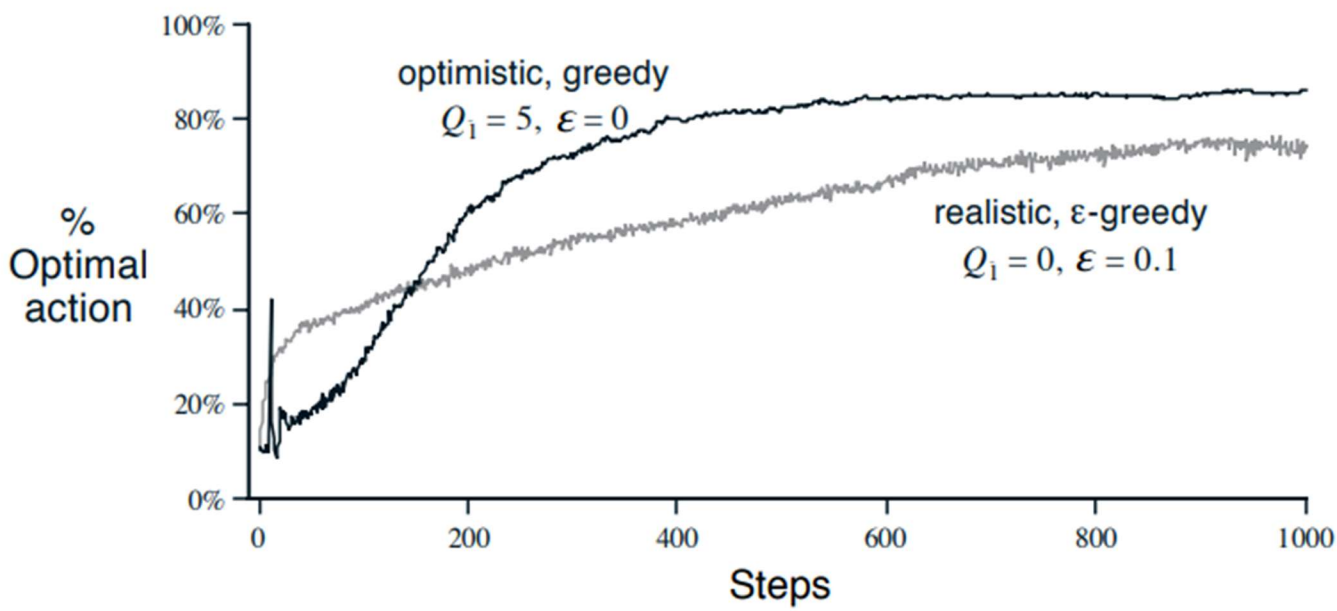
Advantages of Optimistic Initial Values

- Encourages exploration **without using random action selection**.
- Simple and easy to implement.
- Works well for **stationary environments**.
- Does not require an ε parameter for exploration.

Disadvantages of Optimistic Initial Values

- Exploration occurs **only at the beginning** of learning.
- Not suitable for **non-stationary environments** where rewards change over time.
- Requires manual selection of initial values.
- After convergence, the greedy policy may become **stuck with a suboptimal action**.

Feature	Optimistic Greedy Method	ε-Greedy Method
Exploration mechanism	Encouraged by setting **high initial action values**	Exploration done **randomly with probability ε**
When exploration happens	Mostly **at the beginning**	**Throughout the learning process**
Initial performance	**Performs worse initially** due to heavy exploration	Usually **better initially**
Long-term behavior	Exploration **decreases automatically** and becomes greedy	**Exploration never stops**
Overall performance	Often **performs better later** after exploration reduces	**Improves slowly** because random exploration continues



Optimistic Greedy (Top Curve)

- Initially performs **worse**
- Because it explores **a lot**
- Eventually performs **better**
- Exploration decreases over time

ε-Greedy (Bottom Curve)

- Uses **random exploration**
- Exploration **never stops**
- Performance improves **slowly**

Upper Confidence Bound (UCB) Action Selection

In action-value methods, the agent must balance **exploration and exploitation** while selecting actions.
The **greedy method** always selects the action with the highest estimated value, which results in **too much exploitation and no exploration**. This may cause the agent to get stuck with a suboptimal action.

To overcome this problem, the **ε-greedy method** was introduced. In ε-greedy, the agent selects a random action with probability ε and the greedy action with probability $1 - \epsilon$. Although this allows exploration, the exploration is **random** and does not consider how uncertain the action values are.

Therefore, a better strategy is needed where exploration is **directed toward uncertain actions** rather than random ones. One such method is **Upper Confidence Bound (UCB) action selection**.

Idea of UCB

The main idea behind UCB is that **actions with uncertain value estimates should be explored more frequently**. Instead of exploring randomly, UCB selects actions based on both:

1. **Estimated reward (exploitation)**
2. **Uncertainty in the estimate (exploration)**

Thus, UCB tries to explore actions that **might potentially be optimal**.

UCB Action Selection Formula

The UCB method selects the action according to:

$$A_t = \arg\max_a \left[Q_t(a) + c\sqrt{\frac{\ln t}{N_t(a)}} \right]$$

Where:

- $Q_t(a)$ = estimated value of action a (exploitation term)
- $N_t(a)$ = number of times action a has been selected
- t = current time step
- $c > 0$ = parameter controlling the amount of exploration
- $\ln t$ = natural logarithm of time step

The term

$$c\sqrt{\frac{\ln t}{N_t(a)}}$$

is called the **exploration bonus**.

Interpretation of the Formula

The value used to select actions consists of two parts:

1. Estimated value $Q_t(a)$
This encourages **exploitation** by selecting actions with high estimated reward.

2. Exploration bonus $c\sqrt{\frac{\ln t}{N_t(a)}}$
This encourages **exploration** by giving higher bonus to actions that have been tried fewer times.

Thus, UCB selects the action with the **highest upper confidence bound** of the estimated reward.

Behavior of UCB

- If an action is selected many times, $N_t(a)$ increases and the exploration bonus decreases.
- If an action is rarely selected, the exploration bonus is large, encouraging the agent to try it.
- Over time, all actions are explored, but actions with higher rewards are selected more frequently.

If $N_t(a) = 0$, the action is selected immediately because it has maximum uncertainty.

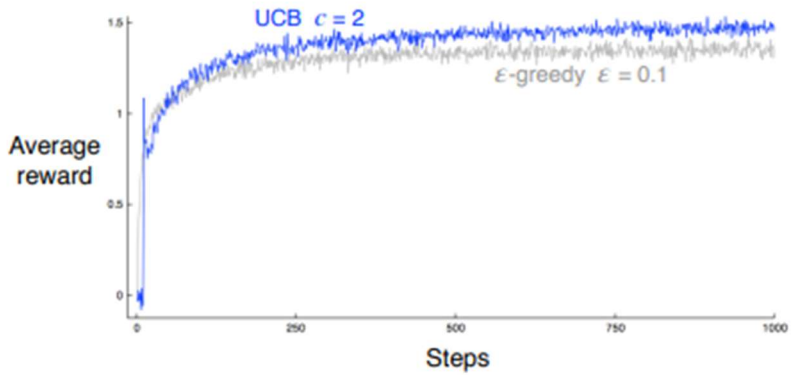
Comparison with ε-Greedy

Feature	ε-Greedy	UCB
Exploration type	Random exploration	Directed exploration
Uses uncertainty	No	Yes
Action selection	Random with probability ε	Based on confidence bound
Efficiency	Less efficient	More efficient in finding optimal action

---

Performance

Experiments on the **10-armed testbed** show that UCB often achieves **higher average reward** than ε-greedy methods. UCB performs slightly worse at the very beginning because it tries all actions initially, but later it learns faster and selects the optimal action more frequently.



Advantages of UCB

- Explores **uncertain actions intelligently**.
- Balances exploration and exploitation effectively.
- Often achieves **higher long-term reward** than ε-greedy.
- Reduces unnecessary random exploration.

Limitations

- Harder to apply in **complex reinforcement learning problems**.
- Difficult to handle **non-stationary environments**.
- Not easy to extend to **large state spaces or function approximation**.

Gradient Bandits

1. Basic Idea

Earlier bandit methods such as **ε-greedy, UCB, and optimistic initial values** estimate the **action value** $Q(a)$ and then choose actions based on those values.

Gradient Bandits follow a **different approach**.
Instead of estimating rewards, they **learn a preference score for each action**.

$$H_t(a)$$

Where:

- $H_t(a)$ = preference for action a at time t

The **higher the preference**, the **more often the action is selected**.

Important point:

- Preferences **do not represent reward values**
- Only the **relative difference between preferences matters**

For example:

Action	Preference
A	2
B	1

Action **A is chosen more often**.

If we add **1000 to both values**, the relative difference remains the same:

Action	Preference
A	1002
B	1001

The probability of choosing actions **does not change**.

2. Softmax Action Selection

Actions are chosen using a **softmax probability distribution**:

$$\pi_t(a) = \frac{e^{H_t(a)}}{\sum_{b-1}^{n} e^{H_t(b)}}$$

Where:

- $\pi_t(a)$ = probability of selecting action a
- $H_t(a)$ = preference of action a

Thus:

$$P(A_t = a) = \pi_t(a)$$

This means:

- Higher preference → higher probability
- Lower preference → lower probability

Example

Action	Preference $H(a)$	$e^{H(a)}$
A	2	7.39
B	1	2.72

Probability of choosing A:

$$P(A) = \frac{7.39}{7.39 + 2.72} = 0.73$$

So **A is chosen about 73% of the time.**

3. Preference Update Rule

Initially, all preferences are equal:

$$H_1(a) = 0$$

So all actions have equal probability.

After choosing action A_t and receiving reward R_t, preferences are updated.

Selected Action Update

$$H_{t+1}(A_t) = H_t(A_t) + \alpha(R_t - \bar{R}_t)(1 - \pi_t(A_t))$$

Non-Selected Actions

$$H_{t+1}(a) = H_t(a) - \alpha(R_t - \bar{R}_t)\pi_t(a)$$

Where:

- α = step size (learning rate)
- R_t = reward received
- $\bar{R}_t$ = average reward (baseline)

4. Role of the Baseline

The **average reward** $\bar{R}_t$ is used as a **baseline**.

The algorithm compares the current reward with the baseline.

Case 1: Reward Above Baseline

$$R_t > \bar{R}_t$$

The action performed **better than average**.

→ Increase its probability.

Case 2: Reward Below Baseline

$$R_t < \bar{R}_t$$

The action performed **worse than average**.

→ Decrease its probability.

Thus:

- Good actions become **more likely**
- Bad actions become **less likely**

5. Gradient Ascent Idea

The algorithm adjusts preferences in a direction that **increases expected reward**.

General gradient ascent rule:

$$H_{t+1}(a) = H_t(a) + \alpha \frac{\partial E[R_t]}{\partial H_t(a)}$$

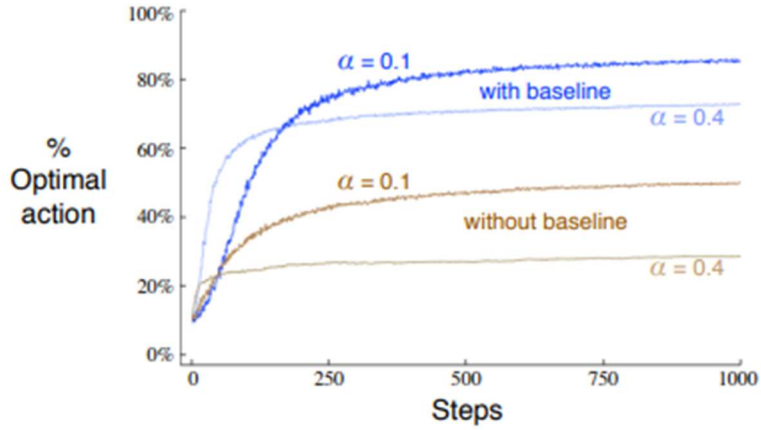
Since the exact gradient is unknown, the algorithm uses **stochastic gradient ascent**, updating preferences using observed rewards.

---

6. Gradient Bandit Algorithm

Steps:

1. Initialize preferences $H(a)$
2. Convert preferences to probabilities using **softmax**
3. Select action based on those probabilities
4. Observe reward
5. Compare reward with baseline
6. Increase probability of good actions
7. Decrease probability of bad actions



7. Performance (Graph Explanation)

Experiments on the **10-armed testbed** show:

- **With baseline** → faster learning and higher performance
- **Without baseline** → slower learning and lower performance

The baseline reduces variance and improves learning stability.